

How to perform 1-dimensional "valid" convolution? [closed]

Asked 10 years, 3 months ago Modified 10 years, 3 months ago Viewed 18k times

Closed. This question needs to be more [focused](#). It is not currently accepting answers.

Update the question so it focuses on one problem only. This will help others answer the question. You can [edit the question](#).

Closed 10 years ago by [Soren](#), [4pie0](#), [László Papp](#), [timgeb](#), [Ivan Ferić](#).

The community reviewed whether to reopen this question 2 years ago and left it closed:

Original close reason(s) were not resolved

(List of close voters is only viewable by users with the close/reopen votes privilege)

[Edit question](#)

I'm trying to implement a 1-dimensional convolution in "valid" mode (Matlab definition) in C++.

It seems pretty simple, but I haven't been able to find a code doing that in C++ (or any other language that I could adapt to as a matter of fact). If my vector size is a power, I can use a 2D convolution, but I would like to find something that would work for any input and kernel.

So how to perform a 1-dimensional convolution in "valid" mode, given an input vector of size I and a kernel of size K (the output should normally be a vector of size $I - K + 1$).

Pseudocode is also accepted.

[c++](#) [math](#) [convolution](#) [Edit tags](#)

Share Edit Follow Reopen Delete Flag

edited Jul 1, 2014 at 21:12



Marco A.

43.5k

27

140

255

asked Jul 1, 2014 at 20:45



Baptiste Wicht

7,623

8

50

112

4 What's the problem? Given the definition of a convolution, this is not really a hard task. – [Nico Schertler](#) Jul 1, 2014 at 21:09



General Question on algos (without source code) may belong over here: [programmers.stackexchange.com/help/on-topic](#) – [Soren](#) Jul 1, 2014 at 21:13



Or actually on this site: [socomp.stackexchange.com/help/on-topic](#) – [Soren](#) Jul 1, 2014 at 21:21



@NicoSchertler Computing the convolution directly from the definition is inefficient and therefore only usable if the size of the problem is not too large.



– [Robert Dodier](#) Jul 1, 2014 at 21:47

3 Answers

Sorted by: [Reset to default](#) Date modified (newest first)

You could use the one of the following implementations:

22

Full convolution:

```
template<typename T>
std::vector<T>
conv(std::vector<T> const &f, std::vector<T> const &g) {
    int const nf = f.size();
    int const ng = g.size();
    int const n = nf + ng - 1;
    std::vector<T> out(n, T());
    for(auto i(0); i < n; ++i) {
        int const jmn = (i >= ng - 1)? i - (ng - 1) : 0;
        int const jmx = (i < nf - 1)? i : nf - 1;
        for(auto j(jmn); j <= jmx; ++j) {
            out[i] += (f[j] * g[i - j]);
        }
    }
    return out;
}
```

f : First sequence (1D signal).

g : Second sequence (1D signal).

returns a `std::vector` of size `f.size() + g.size() - 1`, which is the result of the discrete convolution aka. Cauchy product $(f * g) = (g * f)$.

LIVE DEMO

Valid convolution:

```
template<typename T>
std::vector<T>
conv_valid(std::vector<T> const &f, std::vector<T> const &g) {
    int const nf = f.size();
```

```

int const ng = g.size();
std::vector<T> const &min_v = (nf < ng)? f : g;
std::vector<T> const &max_v = (nf < ng)? g : f;
int const n = std::max(nf, ng) - std::min(nf, ng) + 1;
std::vector<T> out(n, T());
for(auto i(0); i < n; ++i) {
    for(int j(min_v.size() - 1), k(i); j >= 0; --j) {
        out[i] += min_v[j] * max_v[k];
        ++k;
    }
}
return out;
}

```

`f` : First sequence (1D signal).

`g` : Second sequence (1D signal).

returns a `std::vector` of size `std::max(f.size(), g.size()) - std::min(f.size(), g.size()) + 1`, which is the result of the valid (i.e., with out the paddings) discrete convolution aka. Cauchy product $(f * g) = (g * f)$.

LIVE DEMO

Share Edit Follow Flag

edited Jul 2, 2014 at 10:19

answered Jul 1, 2014 at 22:00



Dimitrios Bouzas
42.7k 11 102 176

convolution is not only limited to vector multiplication, `f` & `g` might be a functions – [4pie0](#) Jul 1, 2014 at 22:06

1 @bits_international I believe that the OP refers to discrete convolution (i.e., Cauchy product). – [Dimitrios Bouzas](#) Jul 1, 2014 at 22:12

It works, but it is not a "valid" convolution, but a "full" convolution. – [Baptiste Wicht](#) Jul 2, 2014 at 6:31

@BaptisteWicht please check update, haven't test it though ;) – [Dimitrios Bouzas](#) Jul 2, 2014 at 10:12

I tested it and it indeed works (I have compared with the results on numpy). – [Baptiste Wicht](#) Jul 2, 2014 at 11:27

In order to perform a 1-D valid convolution on an `std::vector` (let's call it `vec` for the sake of the example, and the output vector would be `outvec`) of the size `I` it is enough to create the right boundaries by setting loop parameters correctly, and then perform the convolution as usual, i.e.:

1

```

for(size_t i = K/2; i < I - K/2; ++i)
{
    outvec[i] = 0.0;
    for(size_t j = 0; j < K+1; j++)
    {
        outvec[i - K/2] += invec[i - K/2 + j] * kernel[j];
    }
}

```

Note the starting and the final value of `i`.

Works for any 1-D kernel of any size - provided that the kernel is not of bigger size than vector ;)

Note that I've used the variable `K` as you've described it, but personally I would've understand the 'size' different - a matter of taste I guess. In this example, the total length of the kernel vector is `K+1`. I've also assumed that the `outvec` already has `I - K` elements (BTW: output vector has `I - K` elements, not `I - K + 1` as you have written), so no `push_back()` is needed.

Share Edit Follow Flag

edited Jul 1, 2014 at 21:39

answered Jul 1, 2014 at 21:23



KJMag
2,760 17 16

According to Numpy and Matlab, the result of a "valid" convolution is `I - K + 1`. Otherwise `convolve([1,1,1],[2,2,2])` wouldn't work, but it does (at least in numpy) – [Baptiste Wicht](#) Jul 2, 2014 at 6:44

It would work. In this case: `I = 3`, `K = 2`, and the length of the result vector would be `I - K = 1`, which is correct. Maybe the difference is that you defined `K` differently than `I` in the example above. – [KJMag](#) Jul 2, 2014 at 7:44

There may have been a misunderstanding for the `K`. For me in `convolve([1,1,1],[2,2,2])` `I = 3` and `K = 3` and in which case `I - K = 0` – [Baptiste Wicht](#) Jul 2, 2014 at 8:40

That's what I thought. Anyway, apart from the `K` being defined differently, it should work as expected ;) BTW you don't even have to change the code given above if you define `K` that way; just declare `K` as `int` in your code since in case of `int/int` division the division remainder will be discarded. – [KJMag](#) Jul 2, 2014 at 8:49

I don't understand why you need to implement a convolution function. Doesn't Matlab have a built-in 1D convolution function?

0

Putting that aside, you can implement convolution given a Fourier transform function. You need to be careful about the length of the input and output vectors. The length of the result is `I + k - 1` (not `I - k + 1`, right?). Extend each input vector with zeros to length `N` where `N` is the power of 2 greater than or equal to `I + k - 1`. Take the Fourier transform of the inputs, then multiple the results element by element. Take the inverse Fourier transform of that product, and return the first `I + k - 1` elements (throw the rest away). That's your convolution.

You may need to throw in a scaling factor of `1/N` somewhere since there is no universally-agreed scaling for Fourier transforms, and I don't remember what Matlab assumes for that.



-
- 3 The OP's asking about the C++, not Matlab version. Also, why have you mentioned Fourier transform? Convolution is much more than that. – [KjMag](#) Jul 1, 2014 at 21:30
-
- 2 It is conventional to use FT to efficiently implement convolution, via the identity $\text{FT}(\text{conv}(x, y)) = \text{FT}(x) * \text{FT}(y)$ where the multiplication is taken element by element. – [Robert Dodier](#) Jul 1, 2014 at 21:42
-
- 2 OK, but the result vector in case of valid convolution is smaller than input, so it won't have $I + K - 1$ elements, but $I - K$ instead, because we skip points on the edges, and each edge in 1-D case is of $K/2$ length (provided that the total length of the kernel vector is $K + 1$). – [KjMag](#) Jul 1, 2014 at 21:48
-